TASK 07 – BUILD A MODULE

Aim

To create a Python module containing the milk rate card and calculation functions, and use the module from a separate Python script.

1. Introduction

A module is a Python file that contains reusable code such as variables, functions and classes.

For this task, the file milkrate.py contains the rate constants and the functions rate_per_litre() and payout(). Another Python file imports this module and uses its functions.

2. Concepts

A. Import statements

1. import milkrate

Imports the complete module. Functions are accessed using the module name.

Example:
milkrate.payout(10, 3.5, 8.5)

2. from milkrate import payout

Imports only the payout() function. It can be called directly.

Example:
payout(10, 3.5, 8.5)

3. import milkrate as mr

Imports the module using the shorter alias mr.

Example:
mr.payout(10, 3.5, 8.5)

B. Purpose of if name == "main":

This condition checks whether the Python file is being run directly or imported by another file.

- When milkrate.py is run directly, its test code inside this condition executes.
- When another script imports milkrate, the test code inside this condition does not execute.
- Without this condition, test statements outside functions may run during import.

C. Standard modules

Python provides standard modules that can be imported when required.

- math provides mathematical functions.
- decimal supports decimal arithmetic and accurate money calculations.

D. Where Python searches for modules

Python searches for modules using its module search path, available through sys.path. The current working directory is normally included. The module must be saved in a folder Python can find.

3. Alternative Concept and Complexity Analysis

If the same rate calculation functions were copied into every Python script, each script could work independently. However, when the milk rate changes, every copied program must be updated. This can cause inconsistent calculations.

Using a module is better because the rate card and calculation functions are stored in one place. All scripts can import and reuse them.

Python loads a module the first time it is imported and keeps it in sys.modules. Later imports in the same Python process reuse the loaded module.

For one payment, the calculation takes O(1) time and O(1) extra space. For n farmer records, the total calculation takes O(n) time. Importing a module has a one-time loading cost; it avoids repeatedly maintaining duplicated code.

4. Procedure

1. Create a file named milkrate.py.
2. Define the base rate, fat step value and SNF step value.
3. Complete TODO 1 in rate_per_litre().
4. Complete TODO 2 in payout().
5. Add the if __name__ == "__main__": condition.
6. Run milkrate.py directly and check the three test results.
7. Create a second file named test_milkrate.py.
8. Import the module and call the payout() function.
9. Run the second file and verify the result.

5. Result

The milk rate calculation was organised into a reusable Python module. The module was executed directly and imported successfully from a separate script. The main guard prevented the module's test table from printing during import.